

Université de Liège

Introduction à la calculabilité

 InterEditions

Conseiller scientifique
JACKY AKOKA

© 1991, InterEditions, Paris.

Tous droits réservés. Aucun extrait de ce livre ne peut être reproduit, sous quelque forme ou par quelque procédé que ce soit (machine électronique, mécanique, à photocopier, à enregistrer ou tout autre) sans l'autorisation écrite préalable de l'Éditeur.

ISBN 2-7296-0372-7

Table des matières

Préface	11
1 Introduction	13
1.1 Motivation	13
1.2 Problèmes et programmes	15
1.2.1 La notion de problème	15
1.2.2 La notion de programme	17
1.3 La formalisation des problèmes	19
1.3.1 Alphabets et mots	19
1.3.2 La représentation des problèmes	21
1.3.3 Langages	23
1.4 La description de langages	24
1.4.1 Des opérations sur les langages	24
1.4.2 Les expressions régulières	25
1.5 Les langages non réguliers	29
1.6 Un aperçu de la suite	35
1.7 Exercices	36
2 Les automates finis	37
2.1 Introduction	37
2.2 Description	41
2.3 Formalisation	42
2.4 Représentation et exemples	45
2.5 Les automates finis non déterministes	46
2.5.1 Description	47
2.5.2 Formalisation	48
2.5.3 Exemples	49
2.6 L'élimination du non-déterminisme	50
2.6.1 Principe de la construction	51

2.6.2	Formalisation de la construction	52	4.4.3	Les langages hors-contexte déterministes	125
2.7	Automates finis et expressions régulières	58	4.4.4	Propriétés des langages hors-contexte déterministes	126
2.7.1	Des expressions régulières aux automates	59	4.4.5	Applications	127
2.7.2	Des automates aux langages réguliers	62	4.5	Exercices	128
2.8	Exercices	65			
3	Les grammaires régulières	67	5	Les machines de Turing	129
3.1	Introduction	67	5.1	Introduction	129
3.2	Les grammaires	69	5.2	Définition	130
3.2.1	Définition	69	5.2.1	Description	130
3.2.2	Types de grammaires	72	5.2.2	Formalisation	132
3.3	Les grammaires régulières	76	5.2.3	Langage accepté et langage décidé	137
3.4	Les langages réguliers	78	5.2.4	Autres définitions des machines de Turing	139
3.4.1	Propriétés des langages réguliers	79	5.2.5	Langages récursifs et récursivement énumérables	140
3.4.2	Algorithmes	81	5.3	Thèse de Turing-Church	140
3.5	Au-delà des langages réguliers	83	5.3.1	Extensions des machines de Turing	142
3.5.1	Observations de base	83	5.3.2	Machines à mémoire à accès direct	145
3.5.2	Les théorèmes du "gonflement"	85	5.4	Machines de Turing non déterministes	147
3.5.3	Applications des théorèmes du gonflement	85	5.4.1	Définition	147
3.6	Les applications des langages réguliers	88	5.4.2	Élimination du non-déterminisme	148
3.7	Exercices	89	5.5	Machines de Turing universelles	149
			5.6	Fonctions calculables par une machine de Turing	150
4	Automates à pile et langages hors-contexte	91	5.7	Exercices	151
4.1	Les automates à pile	93			
4.1.1	Description	93	6	Les fonctions récursives	153
4.1.2	Formalisation	94	6.1	Introduction	153
4.1.3	Exemples	98	6.2	Les fonctions primitives récursives	155
4.2	Les langages hors-contexte	100	6.2.1	Définition	155
4.2.1	Définition	100	6.2.2	Exemples	157
4.2.2	Relation avec les automates à pile	103	6.3	Les prédicats primitifs récursifs	162
4.2.3	Propriétés des langages hors-contexte	106	6.3.1	Définition	162
4.3	Au-delà des langages hors-contexte	108	6.3.2	Exemples	163
4.3.1	La notion d'arbre d'analyse	110	6.4	Au-delà des fonctions primitives récursives	167
4.3.2	Le théorème du "gonflement"	114	6.5	Les fonctions μ -récursives	170
4.3.3	Applications du théorème du gonflement	117	6.6	Fonctions μ -récursives et fonctions calculables	172
4.3.4	Algorithmes pour langages hors-contexte	119	6.6.1	Nombres et chaînes de caractères	172
4.4	Les automates à pile déterministes	124	6.6.2	Des fonctions μ -récursives aux machines de Turing	175
4.4.1	Principe	124	6.6.3	Des machines de Turing aux fonctions μ -récursives	176
4.4.2	Formalisation	124	6.7	Fonctions partielles	177

cet ouvrage. Sans eux, je n'aurais pas eu de raison de l'écrire. Certains, notamment Alain Delhez, Bruno Dethier et Eric Lallemand, en ont fait une lecture détaillée et m'ont indiqué des corrections à y apporter. Et, surtout, leurs perplexités occasionnelles ont été pour moi un guide dont ils ne soupçonnaient probablement pas la valeur.

Pierre Wolper

1

Introduction

1.1 Motivation

Les inventeurs de mouvements perpétuels nous font rire. En effet, de telles inventions sont en contradiction avec notre connaissance de la physique. Notre démarche est simple : de connaissances scientifiques de base, nous tirons des conclusions sur ce qui est ou non réalisable en pratique. Cela nous permet d'arriver à une conclusion sans analyse détaillée du prototype de mouvement perpétuel qui nous serait proposé. Nous savons *a priori* qu'une telle réalisation est impossible. Une analyse détaillée sera peut-être nécessaire pour convaincre l'inventeur de son erreur, mais nous sommes convaincus de l'existence d'une faille et ne doutons pas que cette faille puisse être découverte.

En informatique, il existe aussi des réalisations ou des projets de réalisations qui doivent faire rire. Mais quels sont-ils ? Devant le cas trop fréquent du programme énorme qui est "à deux doigts" de fonctionner, que devons-nous faire ?

1. Essayer de le compléter et de le corriger pour qu'il fonctionne correctement ?
2. Revoir la conception du programme et recommencer avec une autre méthode de résolution du problème ?

3. Conclure avec amusement que le problème censé être résolu par le programme n'est en fait soluble par aucun programme, cette conclusion étant tirée sans examen détaillé du programme proposé ?

Le but de cet ouvrage est de nous apprendre à reconnaître les cas qui tombent dans cette troisième catégorie.

D'où proviennent les limites de l'informatique ? La physique impose des limites sur les machines qui peuvent être construites, mais ce ne sont pas ces limites qui vont nous intéresser. Nous allons étudier des limites qui sont indépendantes de la réalisation physique des machines ! Nous allons par exemple démontrer que certains problèmes ne sont pas solubles et ne le seront jamais, quels que soient les développements technologiques futurs. Notre approche sera donc basée sur des principes fondamentaux indépendants de la technologie¹. Ces principes fondamentaux ont commencé à être développés dans les années 1930 (avant l'apparition des ordinateurs) dans le cadre de la logique mathématique. La motivation à cette époque était de donner une définition précise de la notion de preuve formelle. Les développements dans ce domaine se sont poursuivis depuis et se poursuivent encore actuellement.

Si les limites que nous allons établir sont indépendantes de la technologie utilisée pour construire les machines, il est naturel de se demander si elles s'appliquent aussi au cerveau humain. Une autre forme de cette question est de savoir si un cerveau humain peut faire des choses qu'un ordinateur ne peut pas et ne pourra jamais faire. Si l'on considère qu'un cerveau est un ensemble (très complexe) de neurones interconnectés et que chaque neurone a un comportement simple et modélisable, il n'y a pas de raison de croire que les limites que nous allons étudier ne s'appliquent pas au cerveau. Cette conclusion peut heurter et mériterait certainement un débat plus long dans lequel pourraient intervenir des considérations philosophiques. Le lecteur est invité à se pencher sur cette question et à se former une opinion après l'étude de cet ouvrage.

1. Cette situation est semblable à celle des limites sur le rendement des moteurs thermiques déduites des principes de la thermodynamique. Ces limites sont indépendantes de la technologie utilisée pour réaliser les moteurs.

1.2 Problèmes et programmes

La question fondamentale que nous nous posons est de savoir quels problèmes sont solubles par un programme exécuté sur un ordinateur. Pour pouvoir répondre à cette question de façon rigoureuse, nous devons préciser les deux notions qui y interviennent :

- la notion de *problème*,
- la notion de *programme exécuté sur un ordinateur*.

La première de ces notions est relativement simple à formaliser et sera traitée dans les sections suivantes. La formalisation de la deuxième de ces notions est plus délicate et constitue une partie importante du sujet de cet ouvrage. Essayons toutefois de nous en faire, dès à présent, une idée intuitive.

1.2.1 La notion de problème

Commençons par examiner un exemple.

Exemple 1.1

Déterminer si un nombre naturel est pair ou impair est un *problème*. □

Cet exemple nous permet déjà de mettre en évidence plusieurs caractéristiques de la notion de problème.

- Un problème est une question générique, c'est-à-dire qui s'applique à un ensemble d'éléments (dans le cas de l'exemple, les nombres naturels).
- Chaque instance du problème, c'est-à-dire la question posée pour un élément particulier de l'ensemble (est-ce que 35 est pair ?) a une réponse (non).
- La notion de problème est indépendante de la notion de programme. On peut écrire un programme qui résout un problème, mais le problème n'est pas défini par le programme. Par ailleurs, il existe

le plus souvent plusieurs programmes permettant de résoudre un même problème.

Pour résoudre un problème par un programme, il est nécessaire de se fixer une représentation des instances du problème. C'est sur cette représentation qu'opérera le programme.

Exemple 1.2

Les instances du problème de l'exemple 1.1 (les nombres naturels) peuvent être représentées à l'aide de la notation binaire. Un exemple de programme qui résout le problème est alors celui qui examine le dernier chiffre de la représentation, répond *nombre pair* si ce chiffre est 0 et répond *nombre impair* si ce chiffre est 1. Ce problème pourrait aussi être résolu par le programme (un peu idiot, soit) qui convertit la représentation binaire en représentation décimale et répond *nombre pair* si le dernier chiffre de cette représentation est dans l'ensemble $\{0, 2, 4, 6, 8\}$ et *nombre impair* dans les autres cas. $\square$

Considérons maintenant d'autres exemples de problèmes.

Exemple 1.3

- Trier un tableau de nombres est un problème.
- Déterminer si un programme écrit en PASCAL s'arrête quelles que soient les valeurs des données qui lui sont fournies est un problème (problème de l'arrêt).
- Déterminer si un polynôme à coefficients entiers a des racines entières est un problème (dixième problème de Hilbert).

Le premier de ces problèmes est soluble par un programme exécuté sur un ordinateur (méthodes de tri connues). La théorie que nous allons développer dans cet ouvrage montrera que les deux autres ne le sont pas. $\square$

Dans ce qui suit, nous considérerons seulement une classe limitée de problèmes : les problèmes dont la réponse est binaire (oui/non, 0/1). Par exemple, le problème de l'arrêt (exemple 1.3) est un problème à réponse binaire. La classe des problèmes binaires (dont la réponse est binaire) est visiblement une classe limitée, mais elle est suffisante pour étudier les phénomènes qui nous intéressent. De plus, nous verrons comment les résultats obtenus pour la classe des problèmes binaires peuvent être généralisés aux problèmes non binaires (voir section 7.6).

1.2.2 La notion de programme

Notre but ici est de donner une idée intuitive de la différence entre une solution à un problème qui a la forme d'un programme pouvant être exécuté sur un ordinateur, c'est ce que nous appellerons une *procédure effective* (*effective procedure* en anglais), et une solution qui n'a pas cette forme. Un programme écrit en PASCAL est une procédure effective. Pourquoi ? Parce que le programme PASCAL peut être compilé en code machine et que le processeur de la machine est à même d'exécuter ce code en le réduisant à son tour à des opérations élémentaires sur des représentations binaires.

Une autre explication est que le programme PASCAL contient toute l'information nécessaire pour résoudre le problème. Une fois que l'on dispose du programme et des outils qui permettent de l'exploiter, il est possible de résoudre le problème sans la moindre information complémentaire, sans prendre la moindre décision. Tout est précisé par le programme, l'ordinateur ne devra poser aucune question pour exécuter le programme.

Pour mieux comprendre la notion de procédure effective, examinons maintenant un exemple de procédure qui n'est pas effective. Considérons le problème de l'arrêt. La solution "déterminer si le programme n'a pas de boucles ou de séquences d'appels récursifs infinies" n'est pas effective. En effet, après avoir lu cette solution, on se pose la question de savoir comment déterminer si une boucle ou une séquence d'appels récursifs est infinie ? En fait, il n'y a pas de procédure effective qui résout le problème de l'arrêt. Le lecteur qui penserait connaître une telle solution est invité à la tester sur l'exemple suivant :

```

recursive function threen (n: integer):integer;
begin
  if (n = 1) then 1
    else if even(n) then threen(n ÷ 2)
      else threen(3 × n + 1);
end;

```

Pour être complet, il faudrait ajouter qu'un programme PASCAL n'est une procédure effective que s'il s'arrête toujours (donne toujours une réponse quelles que soient les valeurs d'entrée). Cela implique que déterminer si un programme PASCAL est une procédure effective revient à résoudre le problème de l'arrêt. Or ce problème n'est pas soluble par une procédure effective. C'est une circularité de cette forme qui permettra de démontrer l'insolubilité du problème de l'arrêt. Nous reviendrons longuement sur ce point².

La discussion qui précède pourrait mener à penser que nous utiliserons la notion de langage de programmation pour formaliser la notion de procédure effective. Cela ne sera pas le cas, principalement parce qu'un langage de programmation ne définit une procédure effective que par l'intermédiaire d'une procédure d'interprétation ou d'un compilateur. Vu que notre but est de raisonner au sujet des procédures effectives, ce niveau d'indirection compliquerait notre tâche. Toutefois, le fait que cette procédure d'interprétation existe sera suffisant pour que les résultats que nous démontrerons soient applicables aux programmes écrits dans tous les langages de programmation usuels.

Pour formaliser la notion de procédure effective, nous utiliserons en fait des formes de langages de programmation, mais des formes tellement simples que la procédure d'interprétation correspondante est immédiate.

2. Nous avons choisi de distinguer par des caractères plus petits les remarques, exemples ou démonstrations qui ne sont pas indispensables à la compréhension de l'ouvrage. Ces textes qui constituent un deuxième niveau de lecture ont pour but d'informer, d'éclairer, et parfois d'intriguer, le lecteur curieux. Pour plus de clarté, ils ne font cependant pas l'objet d'une numérotation séparée.

Cela nous sera très utile lorsque nous raisonnerons au sujet des comportements possibles de tels programmes. Nous appellerons ces types de programmes dont le mécanisme d'interprétation est immédiat des *automates* (*automata*). Dans ce qui suit, un automate sera donc un *programme* et non pas une machine sur laquelle un programme est exécuté. A chaque classe d'automates (langage de programmation) que nous définirons, sera associé un mécanisme d'exécution (très simple) qui nous permettra de raisonner au sujet de ce qui se passe lors de l'exécution d'un tel automate (programme).

1.3 La formalisation des problèmes

Si un problème doit être résolu par une procédure effective, il faut que ses instances soient représentées d'une façon accessible à cette procédure effective. Quelles sont les représentations possibles ? Si, par exemple, la procédure effective est un programme PASCAL, l'instance du problème (les données sur lesquelles le programme opère) devra être représentée par une collection d'entiers, de chaînes de caractères, ...

Ici, nous voulons adopter une représentation générale faisant abstraction des types de données particuliers utilisés par un langage de programmation. Pour concevoir une telle représentation, examinons ce qui se passe au niveau du code machine. A ce niveau, toutes les données sur lesquelles un programme opère sont représentées par des séquences de bits. Nous allons adopter une légère généralisation de ce point de vue et supposer que chaque ensemble de données (instance de problème) est représenté par une chaîne finie de symboles. L'ensemble des symboles que nous utiliserons dépendra du problème considéré, il pourrait par exemple être l'ensemble des chiffres $\{0, \dots, 9\}$ ou l'ensemble des lettres $\{a, \dots, z\}$.

1.3.1 Alphabets et mots

Reprenons cela de façon plus précise. Pour représenter un problème nous choisissons d'abord un ensemble fini de symboles. Nous appellerons cet ensemble *l'alphabet* (*alphabet*). Nous dénoterons le plus souvent un alphabet par la lettre grecque Σ . Les *symboles* (*symbols*) d'un alphabet,

c'est-à-dire ses éléments, sont fréquemment appelés *caractères* (*characters*).

Définition 1.1

Un *alphabet* est un ensemble fini de symboles. $\square$

Le fait que nous appelons les éléments d'un alphabet des symboles signifie simplement que nous allons associer à chaque élément de l'ensemble une représentation. Le choix de cette représentation est tout à fait arbitraire. La seule caractéristique importante d'un alphabet est sa *taille* (le nombre d'éléments qu'il contient).

Exemple 1.4

Considérons l'alphabet de taille 3. Il importe peu que nous représentions ses symboles par $\{a, b, c\}$, $\{\alpha, \beta, \gamma\}$, $\{1, 2, 3\}$ ou encore $\{\clubsuit, \diamond, \heartsuit\}$. $\square$

L'étape suivante est de définir la notion de *chaîne* (*chain*, *string*) ou *suite* (*sequence*) d'éléments d'un alphabet. Par analogie avec le langage naturel, une suite est souvent appelée *mot* (*word*).

Définition 1.2

Un *mot* défini sur un alphabet est une séquence *finie* d'éléments de cet alphabet. $\square$

Notons qu'un mot a une longueur finie mais arbitraire. Un mot peut aussi bien contenir 3 caractères que 10^{2534} caractères.

Exemple 1.5

- $a, abs, zt, bbbssnbnzzyyyddtrra, ordinateurrapide$ sont des mots sur l'alphabet $\{a, \dots, z\}$.

- $4\clubsuit 3\diamond 5\heartsuit 2\spadesuit, 12765, \clubsuit\heartsuit$ sont des mots sur l'alphabet $\{0, \dots, 7, \clubsuit, \diamond, \heartsuit, \spadesuit\}$.

$\square$

On appellera la *longueur d'un mot* (*length of a word*) le nombre de caractères qu'il contient. Si un mot est dénoté par w , sa longueur sera dénotée par $|w|$. Un mot peut avoir la longueur 0. Dans ce cas, il ne contient aucun caractère et on l'appelle le *mot vide* (*empty word*). Le mot vide est le plus souvent désigné par e, ε ou encore λ .

Pour un mot de longueur n , on parlera souvent du 1^{er} caractère du mot, du 2^e caractère du mot, ..., du n^e caractère. On écrira souvent par exemple

$$w = aaabbaaaabb$$

et $w(1) = a, w(2) = a, \dots, w(11) = b$.

1.3.2 La représentation des problèmes

Il reste maintenant à nous convaincre que nous pouvons représenter les instances d'un problème donné par des mots, c'est-à-dire par des chaînes de caractères. Cela est facile à comprendre. En effet, toutes les données (nombres, chaînes de caractères, ...) utilisées en informatique sont clairement représentables par des chaînes de caractères. Une image est représentable par une chaîne de caractères (la suite des intensités de ses points), un son est aussi représentable par une suite de nombres (numérisation). Même si les instances d'un problème sont décrites en français ou en anglais celles-ci sont représentables par des chaînes de caractères.

Nous supposons donc qu'il est possible de représenter chaque instance des problèmes que nous considérons sous forme d'une chaîne de caractères. Nous appellerons *fonction d'encodage* (*encoding function*) la fonction qui, à partir d'une instance d'un problème, donne son encodage sous la forme d'une chaîne de caractères.

La notion de problème existe indépendamment de toute représentation et de tout encodage. Au niveau le plus abstrait, un problème binaire (oui/non) est un ensemble d'éléments divisé en deux parties : les instances positives et les instances négatives. Par exemple, les instances du problème décrit à l'exemple 1.1 sont les entiers. Les entiers existent indépendamment de leur représentation (unaire, binaire, décimale, hexadécimale ou chiffres romains). Ce qui caractérise l'ensemble des entiers est sa taille (son nombre d'éléments) et une certaine structure que l'on impose sur cet ensemble par l'intermédiaire d'opérations (+, $\times$) et de prédicats ($\leq$). C'est à partir de ces opérations et prédicats que l'on peut définir une partition des entiers en, par exemple, nombres pairs et impairs.

Puisque nous nous intéressons aux procédures effectives et qu'il ne peut y avoir de procédure effective sans encodage des problèmes, nous ne considérerons plus dans ce qui suit que des problèmes dont les instances sont encodées par des mots définis sur un alphabet. Nous ne ferons plus la distinction entre problème et encodage d'un problème.

Soit un problème binaire (les seuls que nous considérons) dont les instances sont encodées par des mots définis sur un alphabet Σ . L'ensemble de tous les mots définis sur Σ peut être partitionné en trois sous-ensembles :

- les mots représentant des instances du problème pour lesquelles la réponse est *oui*, ce sont les *instances positives* (*positive instances*) ;
- les mots représentant des instances du problème pour lesquelles la réponse est *non*, ce sont les *instances négatives* (*negative instances*) ;
- les mots qui ne représentent pas des instances du problème.

On groupe souvent les deux dernières classes et on obtient ainsi une partition de l'ensemble de tous les mots en deux sous-ensembles :

- les mots représentant des instances du problème pour lesquelles la réponse est *oui* (*instances positives*),
- les mots ne représentant pas des instances du problème ou représentant des instances du problème pour lesquelles la réponse est *non* (*instances négatives*).

Si l'on veut vraiment distinguer entre instances négatives du problème et mots qui ne sont pas des instances du problème, il suffit de considérer simultanément le problème (en général immédiat) de reconnaître les mots qui sont des instances (qu'elles soient positives ou négatives) du problème.

1.3.3 Languages

Un problème peut donc être caractérisé par l'ensemble des mots qui sont ses instances positives. Un ensemble de mots est ce que nous appelons un *langage*.

Définition 1.3

Un *langage* (*language*) est un ensemble de mots définis sur le même alphabet. $\square$

On peut donc dire qu'un problème est caractérisé par le langage des encodages de ses instances positives et que résoudre un problème revient à reconnaître les instances positives de ses encodages. C'est pourquoi, dans ce qui suit, nous considérerons toujours le problème de reconnaître des langages, c'est-à-dire de distinguer parmi tous les mots construits sur un alphabet donné ceux qui font partie du langage de ceux qui n'en font pas partie. Nous ne ferons d'ailleurs plus de distinction entre

- résoudre un problème,
- reconnaître le langage des encodages des instances positives du problème.

Exemple 1.6

- $\{aab, aaaa, \varepsilon, a, b, abababababbbbbbbbbbb\}$, $\{\varepsilon, aaaaaaa, a, bbbbbb\}$ et ϕ (l'ensemble vide) sont des langages sur l'alphabet $\{a, b\}$.
- Pour l'alphabet $\{0, 1\}$, $\{0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, \dots\}$ est un langage (le langage contenant tous les mots). De même, l'ensemble de toutes les représentations binaires de nombres pairs est un langage.

- Le langage ϕ (langage vide ne contenant aucun mot) est différent du langage $\{\varepsilon\}$ (langage contenant un mot qui est le mot vide).
- L'ensemble des mots représentant des programmes écrits en PASCAL qui s'arrêtent toujours est un langage.

□

1.4 La description de langages

Nous allons maintenant introduire une notation qui nous permettra de décrire *certain*s langages.

A ce propos, remarquons qu'il n'existe pas de notation qui permet de décrire tous les langages (nous reviendrons sur ce fait à la section 1.5).

Comment peut-on décrire un langage ? Si le langage est fini, il suffit d'énumérer les mots qui en font partie. Si le langage est infini, cela n'est pas possible. La notation que nous allons introduire nous permettra de décrire tous les langages finis et certains langages infinis. Elle est basée sur une représentation directe de certains langages élémentaires (par exemple ceux contenant uniquement un seul mot lui-même formé d'un seul caractère de l'alphabet) et sur la description de langages plus complexes par l'application d'opérations à des langages plus simples.

1.4.1 Des opérations sur les langages

Soit deux langages L_1 et L_2 .

- L'*union* de L_1 et L_2 est le langage contenant tous les mots qui sont soit contenus dans L_1 , soit contenus dans L_2 . Formellement, $L_1 \cup L_2 = \{w \mid w \in L_1 \text{ ou } w \in L_2\}$.
- La *concaténation* de deux langages L_1 et L_2 est le langage contenant tous les mots formés d'un mot de L_1 suivi d'un mot de L_2 . Formellement, $L_1 \cdot L_2 = \{w \mid w = xy, x \in L_1 \text{ et } y \in L_2\}$.

- La *fermeture itérative* d'un langage L_1 ou *fermeture de Kleene* (*Kleene closure* ou *Kleene star*) est l'ensemble des mots formés par une concaténation finie de mots de L_1 . Formellement, $L_1^* = \{w \mid \exists k \geq 0 \text{ et } w_1, \dots, w_k \in L_1 \text{ tels que } w = w_1 w_2 \dots w_k\}$.
- Le *complément* d'un langage est l'ensemble de tous les mots qui ne sont pas dans ce langage. Formellement, $\overline{L_1} = \{w \mid w \notin L_1\}$.

L'ensemble des langages que nous allons définir est appelé l'ensemble des *langages réguliers* (*regular languages*). Il est défini à partir de langages élémentaires et des opérations d'union, de concaténation et de fermeture itérative.

Définition 1.4

L'ensemble $\mathcal{R}$ des *langages réguliers* sur un alphabet Σ est le plus petit ensemble³ de langages satisfaisant les conditions suivantes :

1. $\phi \in \mathcal{R}$; $\{\varepsilon\} \in \mathcal{R}$;
2. $\{a\} \in \mathcal{R}$ pour tout $a \in \Sigma$;
3. si $A, B \in \mathcal{R}$, alors $A \cup B$, $A \cdot B$ et $A^* \in \mathcal{R}$.

□

Dans la définition précédente, spécifier que $\mathcal{R}$ est le plus petit ensemble satisfaisant les conditions énoncées revient à dire que seuls les ensembles construits à partir des ensembles élémentaires ϕ , $\{\varepsilon\}$ et $\{a\}$ par les opérations d'union de concaténation et de fermeture itérative sont dans $\mathcal{R}$.

Pour représenter les langages réguliers, nous allons utiliser une notation qui porte le nom d'*expressions régulières* (*regular expressions*).

1.4.2 Les expressions régulières

Les expressions régulières sont une *notation* pour représenter les langages réguliers. Cette notation suit de près la définition des langages réguliers (définition 1.4). Elle indique simplement comment l'ensemble

3. L'ensemble comportant le moins d'éléments.

régulier dénoté est obtenu à partir des ensembles réguliers élémentaires. Définissons la syntaxe des expressions régulières.

Définition 1.5

Les expressions régulières pour un alphabet Σ sont les expressions formées par les règles suivantes :

1. ϕ , ε et les éléments de Σ sont des expressions régulières,
2. si α et β sont des expressions régulières, alors $(\alpha \cup \beta)$, $(\alpha\beta)$, $(\alpha)^*$ sont des expressions régulières.

□

Les expressions régulières forment elles-aussi un langage au sens où nous l'avons défini. En effet, chaque expression régulière est une chaîne de caractères sur l'alphabet

$$\Sigma' = \Sigma \cup \{(), (, \phi, \cup, *, \varepsilon\}.$$

Ce fait indique que le concept de langage que nous avons défini est utile non seulement pour décrire des encodages de problèmes mais aussi pour définir la syntaxe d'expressions. Cette approche est d'ailleurs celle qui est utilisée pour définir la syntaxe des langages de programmation. Toutefois, avant d'approfondir ce point, il est nécessaire d'étudier plus en détail les formalismes de représentation de langages, ce que nous ferons dans le chapitre 3.

Nous avons donné une définition syntaxique formelle des expressions régulières. Dans la pratique des expressions régulières, nous omettrons les parenthèses lorsque celles-ci ne sont pas nécessaires pour résoudre des ambiguïtés et l'opérateur $*$ sera le plus souvent placé dans la position d'un indice supérieur. Nous écrirons par exemple a^* au lieu de $(a)^*$.

Une expression régulière représente un langage régulier. Lequel ? La réponse à cette question peut paraître évidente et l'est puisqu'on la devine aisément. La voici.

Définition 1.6

Le langage $L(\xi)$ représenté par une expression régulière ξ est défini comme suit :

1. $L(\phi) = \phi$, $L(\varepsilon) = \{\varepsilon\}$,

2. $L(a) = \{a\}$ pour tout $a \in \Sigma$,
3. $L((\alpha \cup \beta)) = L(\alpha) \cup L(\beta)$,
4. $L((\alpha\beta)) = L(\alpha) \cdot L(\beta)$,
5. $L((\alpha)^*) = L(\alpha)^*$.

□

La définition précédente a de quoi faire frémir quelqu'un de méticuleusement rigoureux. En effet, suivant l'endroit où il apparaît, le même symbole peut avoir des significations différentes. Prenons, par exemple, la parenthèse " $()$ ". Elle représente d'une part un symbole de l'alphabet utilisé pour construire les expressions régulières et d'autre part elle est utilisée pour entourer l'argument de l'opérateur L . Similairement $\cup$ représente à la fois un symbole du langage des expressions régulières et un opérateur sur les langages dénotés.

Peut-on éviter cette confusion ? Oui, à condition d'utiliser une notation différente pour chaque signification d'un symbole. On pourrait, par exemple, écrire $L[a] = \{a\}$. On distinguerait ainsi la parenthèse des expressions régulières " $()$ " de celle entourant les arguments de L " $[\]$ ". On dira alors que L et $[\]$ font partie du *métalangage* (*metalanguage*). Le concept de métalangage vient du fait que pour décrire un langage on ne peut éviter l'utilisation d'une notation et donc d'une forme de langage. Le préfixe "méta" que l'on associe à ce langage indique qu'il s'agit en fait du langage d'un langage (du langage servant à décrire un langage) !

En pratique il est immédiat de distinguer ce qui fait partie du métalangage de ce qui fait partie du langage. On utilise donc rarement des notations distinctes pour les symboles de l'un et de l'autre. C'est exactement ce que nous avons fait.

Pour conclure notre discussion des expressions régulières, nous énonçons (sans démonstration) un théorème.

Théorème 1.1

Un langage est régulier si et seulement si il est dénoté par une expression régulière. □

Le théorème 1.1 est évident, c'est pourquoi nous n'avons pas pris la peine de le démontrer. Mais, si l'on veut être rigoureux, comment démontre-t-on l'évidence ? Dans le cas du théorème 1.1 c'est très simple. Le théorème est un "si et seulement si" ; on considère donc chaque direction

séparément. Prenons-en une : si un langage est régulier, il est dénoté par une expression régulière. Il faut donc démontrer que tout langage régulier est dénoté par une expression régulière. La question qui se pose est comment démontrer quelque chose pour tout langage régulier ?

La réponse se trouve dans la forme de la définition des langages réguliers. Cette définition est inductive. On nous dit que certains ensembles de base font partie de $\mathcal{R}$ et puis qu'à partir d'ensembles dans $\mathcal{R}$ il est possible d'en construire de nouveaux par certaines opérations. De plus, tous les ensembles de $\mathcal{R}$ sont obtenus de cette façon. A cette définition inductive on peut associer un principe d'induction que l'on appelle *induction structurale* (*structural induction*). Pour démontrer qu'une propriété p (p pourrait être "il existe une expression régulière dénotant ce langage") est vraie pour tout élément de $\mathcal{R}$, on procède comme suit :

1. on démontre que p est vraie pour les langages ϕ , $\{\varepsilon\}$, et ceux de la forme $\{a\}$;
2. on démontre que si la propriété p est vraie pour les langages A et B , alors p est vraie pour les langages $A \cup B$, $A \cdot B$ et A^* .

Ce principe est en fait la généralisation de l'induction sur les entiers. Cette généralisation devient évidente si l'on considère la définition suivante de l'ensemble des entiers.

- 0 est un entier.
- Si n est un entier, alors $n + 1$ est un entier.

Les éléments que nous venons de donner devraient permettre au lecteur acharné de donner une démonstration complète du théorème 1.1

Considérons maintenant quelques exemples d'expressions régulières.

Exemple 1.7

- L'ensemble de tous les mots définis à partir d'un alphabet $\Sigma = \{a_1, \dots, a_n\}$ est dénoté par l'expression régulière $(a_1 \cup \dots \cup a_n)^*$. Cette expression est souvent abrégée en Σ^* .
- L'ensemble de tous les mots non vides (différents de ε) définis à partir d'un alphabet $\Sigma = \{a_1, \dots, a_n\}$ est dénoté par l'expression régulière $(a_1 \cup \dots \cup a_n)(a_1 \cup \dots \cup a_n)^*$. On écrit souvent cette expression Σ^+ , ou encore⁴ Σ^+ .

4. De façon générale, la notation α^+ représente l'expression régulière $\alpha\alpha^*$.

- Le langage dénoté par l'expression régulière $(a \cup b)^*a(a \cup b)^*$ est le langage des mots composés avec les lettres a et b qui contiennent au moins un a .

□

Exemple 1.8

Montrons que $(a^*b)^* \cup (b^*a)^* = (a \cup b)^*$, c'est-à-dire que les expressions régulières $(a^*b)^* \cup (b^*a)^*$ et $(a \cup b)^*$ dénotent le même ensemble régulier.

Il est clair que $(a^*b)^* \cup (b^*a)^* \subset (a \cup b)^*$ car $(a \cup b)^*$ dénote l'ensemble de tous les mots composés des caractères a et b .

Pour montrer l'inverse, considérons un mot arbitraire

$$w = w_1w_2 \dots w_n \in (a \cup b)^*.$$

On peut distinguer les 4 cas suivants :

1. $w = a^n$ et donc $w \in (\varepsilon a)^* \subset (b^*a)^*$.
2. $w = b^n$ et donc $w \in (\varepsilon b)^* \subset (a^*b)^*$.
3. w contient des a et des b et se termine par b . On a donc $w = \underbrace{a \dots a}_{a^*b} \underbrace{b \dots b}_{(a^*b)^*} \underbrace{a \dots a}_{a^*b} \underbrace{b \dots b}_{(a^*b)^*} b$ ce qui montre que w appartient bien au langage dénoté par $(a^*b)^* \cup (b^*a)^*$.
4. w contient des a et des b et se termine par a . Une décomposition similaire à celle du cas 3 est possible.

□

1.5 Les langages non réguliers

Nous avons mentionné que les expressions régulières permettent de décrire certains langages, mais pas tous. Nous verrons dans le chapitre 2 des

exemples de langages qui ne sont pas réguliers. Ici, nous allons démontrer qu'il existe des langages non réguliers en utilisant une observation fondamentale : *il n'y a pas assez d'expressions régulières pour représenter tous les langages !*

Le raisonnement est très simple. Nous allons comparer le nombre de langages et d'expressions régulières. La seule nouveauté est qu'il y a un nombre infini d'expressions régulières et de langages. Nous devons donc comparer et différencier la taille d'ensembles infinis. Nous verrons que tous les ensembles infinis n'ont pas la même taille !

Avant de nous lancer dans des considérations sur les ensembles infinis, examinons comment on définit la taille d'un ensemble fini. La taille d'un ensemble est le nombre de ses éléments. Cette définition satisfait notre intuition mais repose sur notre connaissance des nombres (naturels) et de leurs propriétés. N'étant pas aussi apte à manipuler l'infini, il nous faut une définition plus directe.

La manière d'y arriver est de donner un critère permettant de déterminer quand deux ensembles ont la même taille. Ce critère est simplement que les éléments des deux ensembles peuvent être mis en correspondance un à un. En d'autres termes, il existe une bijection entre les ensembles.

Exemple 1.9

Les ensembles $\{0, 1, 2, 3\}$, $\{a, b, c, d\}$, $\{\clubsuit, \diamond, \heartsuit, \spadesuit\}$ ont tous la même taille. Ils peuvent être mis en bijection, par exemple $\{(0, \clubsuit), (1, \diamond), (2, \heartsuit), (3, \spadesuit)\}$. $\square$

Si deux ensembles peuvent être mis en bijection nous dirons qu'ils ont la même *cardinalité* (*cardinality*). Cardinalité est le terme technique utilisé en théorie des ensembles pour désigner la taille d'un ensemble.

On distingue deux mesures de la taille d'un ensemble, la cardinalité et l'ordinalité. Comme nous venons de le voir, la cardinalité est définie par l'intermédiaire de bijections entre ensembles. L'ordinalité n'est définie que pour des ensembles ordonnés (sur lesquels on a défini une relation d'ordre). On dira que deux ensembles ont la même ordinalité s'il existe entre eux une bijection qui préserve ces relations d'ordre. Cela signifie que si $a \leq b$ dans le premier ensemble et que les images de a et b dans

le deuxième ensemble sont respectivement a' et b' , on doit avoir $a' \leq b'$ dans le deuxième ensemble.

Pour les ensembles finis, l'ordinalité et la cardinalité coïncident (lorsque l'on a une bijection entre ensembles, on a une bijection qui respecte l'ordre). Pour les ensembles infinis, ce n'est pas du tout le cas, mais nous n'en dirons pas plus ici.

Maintenant que nous pouvons comparer la cardinalité de deux ensembles, il est naturel de regrouper en classes les ensembles de même cardinalité.

La relation "avoir la même cardinalité" permet de regrouper les ensembles en classes vu qu'elle est une *relation d'équivalence* (*equivalence relation*). Cela signifie qu'elle a les trois caractéristiques suivantes :

- elle est réflexive ($|A| = |A|$) vu qu'un ensemble peut toujours être mis en bijection avec lui-même,
- elle est symétrique (si $|A| = |B|$ alors $|B| = |A|$) vu que l'existence d'une bijection est symétrique,
- elle est transitive (si $|A| = |B|$ et $|B| = |C|$ alors $|A| = |C|$) vu que deux bijections peuvent être composées pour donner une bijection.

Une relation d'équivalence définit une partition d'un ensemble, c'est-à-dire qu'elle répartit les éléments de l'ensemble en sous-ensembles disjoints dont l'union est égale à tout l'ensemble.

Exemple 1.10

Sur les entiers, l'égalité modulo 7 est une relation d'équivalence qui définit une partition des entiers en 7 sous-ensembles : les multiples de 7, les multiples de 7 plus 1, les multiples de 7 plus 2, ... $\square$

La cardinalité d'un ensemble est donc caractérisée par la classe d'ensembles de même cardinalité auquel il appartient. Pour pouvoir parler simplement de ces classes, on les désigne par un de leurs éléments. Pour les cardinalités finies, les éléments représentatifs sont les ensembles de la forme

$$\{0, 1, 2, \dots, n-1\}$$

que l'on dénote simplement par n .

Voilà une bien longue discussion pour en arriver à dire que la cardinalité d'un ensemble comportant n éléments est n , mais cette discussion nous prépare à examiner le cas des ensembles infinis.

Le premier et le plus petit des ensembles infinis est celui des nombres naturels $\{0, 1, 2, \dots\}$. Utilisant le concept de bijection, nous pouvons définir les ensembles de même taille que celui-ci.

Définition 1.7

Un ensemble infini est *dénombrable*⁵, *énumérable* (*denumerable*, *countable*) s'il existe une bijection entre cet ensemble et l'ensemble des nombres naturels. $\square$

La cardinalité des ensembles dénombrables est habituellement dénotée par $\aleph_0$ ($\aleph$ est la première lettre de l'alphabet hébraïque, elle se lit "aleph"). Beaucoup d'ensembles sont dénombrables (ont la cardinalité $\aleph_0$). Examinons des exemples.

Exemple 1.11

1. L'ensemble des nombres pairs est dénombrable. La bijection avec les naturels est

$$\{(0, 0), (2, 1), (4, 2), (6, 3), \dots\}.$$

Une généralisation de cette observation est que tout sous-ensemble infini des naturels est dénombrable.

2. L'ensemble des mots sur l'alphabet $\{a, b\}$ est dénombrable. Pour obtenir une énumération (bijection avec les naturels) des mots sur l'alphabet $\{a, b\}$ on classe les mots par ordre de taille croissante et, pour les mots d'une même taille, par ordre lexicographique (l'ordre du dictionnaire). Cela donne donc

$$\{(\varepsilon, 0), (a, 1), (b, 2), (aa, 3), (ab, 4), (ba, 5), (bb, 6), (aaa, 7) \dots\}.$$

5. Nous donnons ici un sens technique précis à dénombrable qui ne s'applique qu'à des ensembles infinis. Il est clair qu'au sens intuitif, tout ensemble fini est dénombrable puisque qu'on peut le mettre en bijection avec un préfixe fini de l'ensemble des naturels.

3. Les nombres rationnels sont dénombrables. En effet, un nombre rationnel est caractérisé par une paire de naturels a/b , telle que $b \neq 0$ et telle que a et b n'ont pas de facteurs communs. Classons les paires de naturels satisfaisant ces restrictions suivant l'ordre de la somme $a + b$ et, pour chaque valeur de la somme, par ordre de numérateur croissant. Cela nous donne une bijection avec les naturels

$$\{(0/1, 0), (1/1, 1), (1/2, 2), (2/1, 3), (1/3, 4), (3/1, 5), \dots\}.$$

4. Les expressions régulières sont dénombrables. En effet, les expressions régulières sont des chaînes de caractères sur un alphabet fini. L'ensemble de toutes les chaînes de caractères sur un alphabet est dénombrable (voir exemple 2 ci-dessus). Les expressions régulières sont un sous-ensemble infini de ces chaînes de caractères et sont donc aussi dénombrables (voir exemple 1 ci-dessus).

$\square$

Après les exemples que nous venons de considérer, on pourrait se demander si tous les ensembles infinis n'ont pas la cardinalité $\aleph_0$. Cela n'est pas le cas. Il y a des ensembles infinis qui ont une cardinalité supérieure à $\aleph_0$, par exemple l'ensemble des sous-ensembles d'un ensemble dénombrable. Démontrons-le.

Théorème 1.2

L'ensemble des sous-ensembles d'un ensemble dénombrable n'est pas dénombrable. $\square$

Démonstration

La démonstration de ce théorème repose sur une technique simple mais très puissante : *la technique de la diagonale* (*diagonalisation*). Soit un ensemble dénombrable

$$A = \{a_0, a_1, a_2, \dots\}$$

et soit S l'ensemble de ses sous-ensembles. Supposons que S est dénombrable et donc que

$$S = \{s_0, s_1, s_2, \dots\}.$$

Construisons le tableau infini suivant :

	a_0	a_1	a_2	a_3	a_4	$\dots$
s_0	×	×		×		
s_1	×	□		×		
s_2		×	×		×	
s_3	×		×	□		
s_4		×		×	□	
$\vdots$						

Ce tableau indique quels éléments de A appartiennent à chaque élément de S . Chaque ligne du tableau correspond à un élément s_j de S et comporte une croix dans la colonne correspondant à a_i si $a_i \in s_j$. Le tableau est donc simplement une représentation graphique du contenu des ensembles s_j .

Considérons maintenant l'ensemble $D = \{a_i \mid a_i \notin s_i\}$.

Cet ensemble est celui défini par les symboles $\square$ se trouvant sur la diagonale de notre tableau. Visiblement cet ensemble est un sous-ensemble de A (il contient tous les éléments a_j de A tels que le symbole $\square$ se trouve à l'intersection de la diagonale du tableau avec la colonne a_j).

L'ensemble D existe donc bien, mais il ne peut être un des ensembles s_i . En effet, supposons que $D = s_k$. Cela est impossible car $a_k \in D$ si et seulement si $a_k \notin s_k$. Nous avons donc une contradiction. Ce qui est contredit est l'hypothèse faite au début de cette démonstration, à savoir que l'ensemble des sous-ensembles de A était dénombrable. $\square$

Le théorème 1.2 est une des causes des limites fondamentales de l'informatique que nous allons étudier. Il implique par exemple que tous les langages ne peuvent être réguliers. En effet,

- l'ensemble des langages est l'ensemble des sous-ensembles d'un ensemble dénombrable (l'ensemble des mots) et n'est donc pas dénombrable (théorème 1.2) ;

- l'ensemble des langages réguliers est dénombrable puisque chaque langage régulier est dénoté par une expression régulière et que l'ensemble des expressions régulières est dénombrable ;
- il y a donc (beaucoup) plus de langages que de langages réguliers.

Le résultat que nous venons d'établir a une portée très générale et nous permettra d'établir qu'il existe des problèmes insolubles par une procédure effective. En effet, le nombre de problèmes est égal au nombre de langages et est donc non dénombrable. Par contre, une procédure effective devant être décrite par une chaîne finie, il n'existe qu'un nombre dénombrable de procédures effectives. Il ne peut donc y avoir une procédure effective pour chaque problème !

1.6 Un aperçu de la suite

Dans les chapitres suivants, nous commencerons par rechercher une définition de la notion de procédure effective. Nous examinerons successivement diverses classes d'automates qui pourraient servir de définition à cette notion. Notre but est de montrer que la notion de procédure effective à laquelle nous arriverons est naturelle. De plus, les différentes classes d'automates que nous étudierons sont aussi intéressantes en elles-mêmes et ont de nombreuses applications en informatique. Cela motive d'autant plus leur étude.

Nous passerons ensuite à l'étude de problèmes qui ne sont pas solubles algorithmiquement et nous examinerons les techniques qui permettent de démontrer qu'un problème n'est pas soluble. Nous donnerons des exemples concrets de problèmes pour lesquels il n'existe pas de procédure effective.

La dernière partie de cet ouvrage est consacrée à des limites plus quantitatives de l'informatique. Au lieu de nous intéresser simplement à ce qui est soluble ou non, nous affinerons notre étude et nous examinerons quels problèmes sont ou ne sont pas solubles *efficacement*, c'est-à-dire avec une consommation "raisonnable" de ressources (temps et espace mémoire). Nous arriverons à établir qu'il existe des problèmes qui, bien qu'étant solubles par une procédure effective, ne le sont pas avec une efficacité

qui permette d'espérer une implémentation de leur solution utilisable en pratique.

1.7 Exercices

1.1 Démontrer que pour tout langage L , $L^* = (L^*)^*$.

1.2 Montrer qu'il existe des langages L_1 et L_2 tels que $(L_1 \cup L_2)^* \neq L_1^* \cup L_2^*$.

1.3 Montrer qu'il existe des langages L_1 et L_2 tels que $(L_1 \cdot L_2)^* \neq L_1^* \cdot L_2^*$.

1.4 Pour chacun des langages suivants, donner une expression régulière représentant son complément.

1. $(a \cup b)^*b$
2. $((a \cup b)(a \cup b))^*$

1.5 Montrer que chacune des expressions régulières suivantes représente le langage $(a \cup b)^*$.

1. $(a^*b^*)^*$
2. $\phi \cup \varepsilon \cup a(a \cup b)^* \cup b(a \cup b)^*$
3. $a^* \cup (a^*b^*)^*b(a \cup b)^*$

1.6 Démontrer que tout sous-ensemble infini d'un ensemble infini dénombrable est dénombrable.

1.7 Démontrer que l'union de deux ensembles infinis dénombrables est un ensemble dénombrable.

1.8 Démontrer que l'ensemble des sous-ensembles *finis* de l'ensemble des nombres naturels est dénombrable. Généraliser à un ensemble infini dénombrable quelconque.

2

Les automates finis

2.1 Introduction

Dans ce chapitre nous allons étudier une première classe d'automates : les *automates finis* (*finite-state automata*). Le concept d'automate fini nous aidera à cerner la notion de procédure effective, mais ne sera pas notre choix final de modélisation pour cette notion. Nous verrons aussi, qu'indépendamment de leur relation avec la notion de procédure effective, les automates finis peuvent être très utiles dans la résolution de certains problèmes informatiques, par exemple les problèmes de recherche de chaînes de caractères dans un texte.

Pour bien comprendre en quoi le concept d'automate fini est une modélisation de la notion de procédure effective, nous allons tenter de le dériver de la notion de programme exécuté sur un ordinateur.

Considérons un ordinateur qui exécute un programme fixé. On peut supposer qu'il est composé

- d'un processeur auquel sont attachés un certain nombre de registres (dont le *compteur de programme* (*program counter*), habituellement dénoté PC),
- d'une mémoire contenant le programme (comme celui-ci est fixé, cette mémoire pourrait être une mémoire morte ou ROM),

- d'une mémoire servant à stocker les données.

Supposons qu'au départ les données à traiter se trouvent dans la mémoire de l'ordinateur. On peut alors schématiser l'exécution du programme par une succession de cycles composés comme suit.

1. L'instruction indiquée par le compteur de programme est transférée de la mémoire vers le processeur.
2. Le processeur exécute cette instruction, ce qui a pour effet de modifier le contenu des registres et/ou de la mémoire de données.
3. Le compteur de programme est incrémenté (ou modifié si l'instruction exécutée est une instruction de branchement).

Comment peut-on décrire cette succession de cycles d'exécution de façon plus abstraite? Pour pouvoir déterminer l'effet d'un cycle, il suffit de connaître le contenu de la mémoire de données et des registres (dont le PC). A l'aide de cette information, on peut sans ambiguïté déterminer ce que sera le contenu de la mémoire et des registres après l'exécution d'un cycle (d'une instruction). On peut donc voir l'exécution d'une instruction comme une transformation du contenu de la mémoire et des registres.

Nous appellerons le contenu de la mémoire et des registres de l'ordinateur son *état (state)*. Chaque contenu de la mémoire et des registres définit un état différent.

Le concept d'état d'un système qu'il soit informatique ou autre est un concept très général et très utile. En termes simples, l'état d'un système à un moment donné est toute l'information nécessaire pour prédire son évolution future. Il s'agit en fait d'un résumé de son histoire suffisant pour connaître son évolution future.

Par exemple dans le cas d'un système dynamique, connaître la position et la vitesse de chaque élément est suffisant pour pouvoir déterminer l'évolution future du système. Il n'est pas nécessaire de savoir comment chaque élément a acquis sa position et sa vitesse pour déterminer ce qui va se passer ensuite.

Similairement, dans le cas d'un ordinateur, le contenu de la mémoire et des registres est suffisant pour déterminer ce qui va se passer. Il n'est pas nécessaire de connaître la séquence d'instructions qui a abouti à un contenu donné de la mémoire et des registres pour connaître la suite de l'exécution.

Chaque cycle transforme donc l'état de la machine. Cette transformation ne dépend que du programme et de la machine. Si ces deux éléments sont fixés comme nous le supposons, cette transformation peut donc être vue comme une *fonction* de l'ensemble des états de la machine vers lui-même. Nous appellerons cette fonction la *fonction de transition (transition function)*. Connaître cette fonction et l'état dans lequel la machine se trouve au départ, l'*état initial (initial state)*, est suffisant pour savoir exactement ce qui va se passer durant l'exécution du programme par la machine. Remarquons enfin que, sur une machine donnée, le nombre d'états possibles est fini et fixé *a priori*, bien qu'en général énorme.

Exemple 2.1

Une machine contenant 8 Mbytes de mémoire et 16 registres de 32 bits a un nombre d'états possibles égal à $2^{26+2^9} = 2^{67109376}$. □

Est-il vraiment légitime de raisonner au sujet d'une machine possédant $2^{67109376}$ états comme si elle comportait un nombre fini d'états? En principe, on peut examiner tous les éléments d'un ensemble fini. En pratique, on n'arrivera jamais au bout d'un ensemble comportant $2^{67109376}$ éléments. Quel point de vue faut-il adopter? Dans ce chapitre, nous considérons que le nombre d'états est fini, mais nous reviendrons sur ce choix.

Nous arrivons donc à une première modélisation d'un programme exécuté par une machine. Un programme exécuté par une machine est modélisé par

- un ensemble *fini* d'états,
- une fonction définie sur cet ensemble d'états,
- un état initial.

Une exécution du programme par la machine est représentée par la séquence d'états obtenus par applications successives de la fonction de transition à l'état initial.

L'hypothèse suivant laquelle le nombre d'états possibles est fini a des conséquences très fortes. Elle implique que, pour chaque état initial, on peut complètement déterminer quelle sera la séquence des états. Cette

séquence pourrait être infinie, mais comme le nombre d'états est fini, un même état apparaîtra fatalement deux fois dans la séquence. A partir de ce moment, la partie de séquence comprise entre les deux apparitions de cet état se répétera indéfiniment et il n'est pas nécessaire de calculer la suite de la séquence.

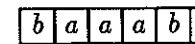
De plus, notre modélisation implique que le nombre d'états initiaux possibles et donc le nombre de données différentes qui peuvent être traitées est fini et fixé. Or, nous avons modélisé les problèmes par des langages. Cela signifie qu'une procédure effective doit pouvoir être appliquée à une infinité de mots différents, ce qui n'est pas possible dans notre modèle. Nous devons donc l'adapter, pour que des mots de longueur arbitraire puissent être traités.

Nous pourrions supposer que la mémoire de la machine est infinie. C'est une hypothèse que nous adopterons dans des chapitres ultérieurs. Ici, nous voulons maintenir notre hypothèse d'un nombre fini d'états et étudier les conséquences de ce choix. Nous allons donc simplement adapter la façon dont le mot à traiter est mis à la disposition du programme. Plutôt que de supposer qu'il est initialement dans la mémoire de la machine, nous allons supposer qu'il est fourni à la machine caractère par caractère.

Supposons donc que la machine possède un dispositif de lecture qui peut consulter à chaque instant le caractère suivant du mot à traiter. Le seul point à éclaircir pour adapter notre modèle à cette situation est de déterminer quand on passe au caractère suivant. Nous allons supposer que cela se fait à chaque cycle.

- Si les caractères sont fournis plus lentement que les cycles de la machine, on peut supposer que celle-ci est bloquée jusqu'à ce que le caractère suivant soit disponible.
- Dans notre modélisation, un cycle équivaut à une application de la fonction de transition. S'il faut plusieurs cycles pour traiter un caractère, ceux-ci peuvent être remplacés par un cycle unique. En effet, il suffit de redéfinir la fonction de transition pour qu'elle modélise cette succession de cycles par une seule transition. Aussi, comme nous l'avons déjà mentionné, dans l'hypothèse d'un nombre fini d'états les exécutions infinies sont sans intérêt. On peut donc

ruban :



tête :

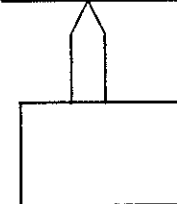


Figure 2.1 - Automate fini

toujours supposer que le traitement de chaque caractère se fait en un nombre fini de cycles.

- Pour la même raison, nous pouvons supposer qu'après avoir lu le dernier caractère, la machine s'arrête et fournit sa réponse.

Nous arrivons ainsi au concept d'automate fini que nous décrivons dans la section suivante.

2.2 Description

Un *automate fini déterministe*¹ (*deterministic finite-state automaton* ou *deterministic finite automaton*) se compose des éléments suivants.

- Un *ruban d'entrée* (*input tape*) sur lequel on place le mot à traiter. Ce ruban est divisé en cases et chaque case contient un seul caractère du mot à traiter appelé le *mot d'entrée* (*input word*). L'automate possède une *tête de lecture* (*read head*) qui indique la position du caractère suivant à lire dans le mot d'entrée (voir figure 2.1).
- Un *ensemble d'états* (*set of states*). Parmi ces états, nous distinguerons :
 - un *état initial* (*initial state*) qui est l'état de l'automate au début de l'exécution,

1. Par opposition aux automates finis non déterministes que nous étudierons à la section 2.5.

– des *états accepteurs* (*accepting states*) qui serviront à définir les mots que l'automate accepte (c'est-à-dire ceux pour lesquels il répond "oui").

- Une *fonction de transition* (*transition function*). Cette fonction indique pour chaque état et symbole lu quel est l'état suivant de l'automate.

L'exécution d'un automate fini sur un mot donné peut se décrire comme suit.

- Au départ, la machine se trouve dans l'état initial et le mot d'entrée est placé sur le ruban. La tête de lecture se trouve devant le premier caractère du mot d'entrée.
- A chaque étape de son exécution, l'automate lit un symbole du ruban, détermine l'état suivant au moyen de la fonction de transition puis avance la tête de lecture vers la case suivante du ruban.
- L'automate s'arrête lorsque toute la chaîne a été lue. Le mot est accepté (l'automate répond qu'il fait partie du langage) si la machine se trouve alors dans un état accepteur.

2.3 Formalisation

Plus formellement, nous pouvons dire qu'un automate fini déterministe est défini par un quintuplet $M = (Q, \Sigma, \delta, s, F)$, où

- Q est un ensemble fini d'états,
- Σ est un alphabet,
- $\delta : Q \times \Sigma \rightarrow Q$ est la fonction de transition,
- $s \in Q$ est l'état initial,
- $F \subseteq Q$ est l'ensemble des états accepteurs.

La définition précédente décrit simplement comment un automate fini est constitué. Nous devons encore dire ce qu'un tel automate représente. Intuitivement, un automate est une machine qui résout un problème,

c'est-à-dire qui reconnaît (accepte) un certain langage. Donner une signification à un automate revient donc à définir le langage qu'il accepte.

La discussion des sections précédentes nous donne déjà une idée intuitive de ce qu'est le langage accepté par un automate. Essayons d'en donner une définition formelle. Pour cela nous introduisons la notion de *configuration*. Une configuration d'un automate fini représente l'information nécessaire pour déterminer son évolution future. Elle se compose d'une paire reprenant l'état dans lequel l'automate se trouve et la partie du mot d'entrée restant à traiter. Formellement, une configuration est donc un élément $(q, w) \in Q \times \Sigma^*$.

La configuration d'un automate fini est en fait son *état* au sens où nous l'avons défini dans la remarque de la page 38. L'état d'un automate fini n'est donc pas à proprement parler son *état*.

Chaque étape de l'exécution d'un automate modifie sa configuration. Nous dirons qu'une configuration est *dérivable en une étape* (*derivable in one step*) d'une autre si elle est obtenue de cette dernière par une étape de l'exécution de l'automate.

Définition 2.1

La configuration (q', w') est dérivable en une étape de la configuration (q, w) par la machine M (nous écrirons $(q, w) \vdash_M (q', w')$) si

- $w = \sigma w'$ (le premier caractère de w est σ et w' est w dont on a retiré le premier caractère),
- $q' = \delta(q, \sigma)$ (q' est le résultat de l'application de la fonction de transition à q et à σ).

□

Nous dirons également qu'une configuration est *dérivable*² (*derivable*) d'une autre si elle peut être obtenue de cette dernière par une séquence finie (éventuellement de longueur nulle) de dérivations en une étape.

2. Sous-entendu en plusieurs étapes.

Définition 2.2

La configuration (q', w') est dérivable de la configuration (q, w) par la machine M (nous écrirons $(q, w) \vdash_M^* (q', w')$) si

- il existe $k \geq 0$ et des configurations (q_i, w_i) , $0 \leq i \leq k$ telles que
- $(q, w) = (q_0, w_0)$,
- $(q', w') = (q_k, w_k)$,
- pour tout $0 \leq i < k$, $(q_i, w_i) \vdash (q_{i+1}, w_{i+1})$.

□

Finalement, nous pouvons définir l'exécution d'un automate (*the execution of an automaton*) sur un mot w .

Définition 2.3

L'exécution d'un automate sur un mot w est la suite des configurations

$$(s, w) \vdash (q_1, w_1) \vdash (q_2, w_2) \vdash \dots \vdash (q_n, \varepsilon)$$

où s est l'état initial et ε le mot vide. □

Remarquons que pour le type d'automate que nous considérons ici (déterministe) chaque mot w définit une seule exécution.

Nous dirons qu'un mot w est accepté par un automate si le dernier état de l'exécution de l'automate sur le mot w est un état accepteur.

Définition 2.4

Un mot w est accepté par un automate M si

$$(s, w) \vdash_M^* (q, \varepsilon)$$

et $q \in F$. □

Exemple 2.2

Soit un automate $M = (Q, \Sigma, \delta, s, F)$ dont l'alphabet est $\Sigma = \{a, b\}$ et soit $w = ababa$. L'exécution de l'automate sur le mot w est une séquence de la forme

$$(s, ababa) \vdash (q_1, baba) \vdash (q_2, aba) \vdash (q_3, ba) \vdash (q_4, a) \vdash (q_5, \varepsilon).$$

Le mot d'entrée $ababa$ sera accepté si l'état q_5 est accepteur, c'est-à-dire $q_5 \in F$. □

Le langage accepté par un automate M est l'ensemble des mots acceptés par cet automate. Il est noté $L(M)$.

Définition 2.5

Le langage accepté par un automate M est l'ensemble des mots $L(M)$ tel que

$$L(M) = \{w \in \Sigma^* \mid (s, w) \vdash_M^* (q, \varepsilon) \text{ avec } q \in F\}.$$

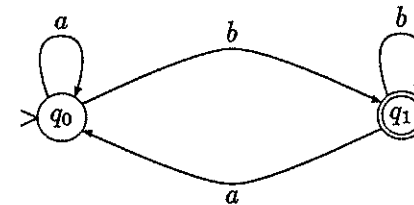
□

2.4 Représentation et exemples

Un automate fini est entièrement défini par le quintuplet $(Q, \Sigma, \delta, s, F)$. Toutefois, en pratique, on représentera souvent un automate fini au moyen d'un graphe. Chaque état de l'automate est représenté par un sommet du graphe. La relation de transition est représentée par des arcs étiquetés. Si $\delta(p, \sigma) = q$, le graphe comporte un arc étiqueté par σ reliant les sommets p et q . Par convention, l'état initial est repéré par une pointe de flèche, et les états accepteurs par un double cercle.

Exemple 2.3

Le graphe



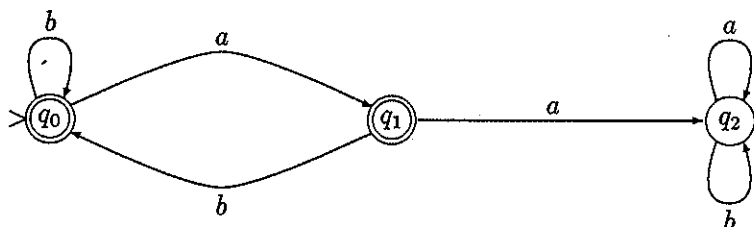
représente l'automate défini par

$\delta:$	q	σ	$\delta(q, \sigma)$	$Q = \{q_0, q_1\}$
	q_0	a	q_0	$\Sigma = \{a, b\}$
	q_0	b	q_1	$s = q_0$
	q_1	a	q_0	$F = \{q_1\}$
	q_1	b	q_1	

Cet automate accepte l'ensemble des mots qui se terminent par b . $\square$

Exemple 2.4

Le langage accepté par l'automate



est

$\{w \mid w \text{ ne contient pas 2 } a \text{ consécutifs}\}.$

$\square$

2.5 Les automates finis non déterministes

Dans cette section nous allons considérer une généralisation des automates finis. Cette généralisation permettra à l'automate de *choisir* une exécution parmi plusieurs. Cette possibilité est en contradiction avec notre intuition de la notion de procédure effective. Alors, pourquoi la considérer ? Il y a plusieurs raisons.

1. Lorsque l'on considère une modélisation de la notion de procédure effective, il est important de se poser des questions du genre : "Et si on ajoutait la possibilité $X \dots$?" L'intérêt de ce type de question est de voir si notre modélisation est robuste, c'est-à-dire si de petites variations ne modifient pas les conclusions que l'on peut en

tirer. Par extension, il est aussi intéressant de voir les conséquences de modifications plus profondes. Le lecteur sera peut-être surpris d'apprendre que nous démontrerons que le non-déterminisme ne modifie pas la classe des langages qui sont acceptés par un automate fini.

2. Indépendamment de leur relation avec la notion de procédure effective, les automates finis sont intéressants comme formalisme de description de langages. Or, il se fait que l'introduction du non-déterminisme facilite nettement la description de certains langages.
3. Le concept de non-déterminisme nous sera utile tout au long de cet ouvrage. Les automates finis sont un contexte simple dans lequel il peut être introduit.

2.5.1 Description

Les automates finis non déterministes sont des automates finis où l'on permet :

- plusieurs transitions correspondant à la même lettre dans chaque état,
- des transitions sur le mot vide (c'est-à-dire sans avancer dans le mot d'entrée),
- des transitions sur des mots de longueur supérieure à 1 (regroupement de transitions).

On voit déjà à partir de cette définition qu'un automate non déterministe aura en général plusieurs exécutions sur un même mot. Pour que ce mot soit accepté il suffira qu'une seule de ces exécutions mène à un état accepteur. De façon imagée, on peut dire que l'on laisse à l'automate le soin de choisir la "bonne" exécution.

Insistons encore sur le fait que les automates finis non déterministes ne définissent pas en tant que tels une procédure de calcul. Ils représentent seulement un formalisme utile pour la définition de langages. L'utilité du non-déterminisme vient de ce qu'il permet de représenter très simplement la disjonction. Construire un automate non déterministe qui accepte tous les mots acceptés par un automate A ou par un automate B est

très simple. Il n'en irait pas de même de la construction d'un automate déterministe.

2.5.2 Formalisation

Formellement, un automate fini non déterministe se définit presque comme un automate fini déterministe. La différence est que nous avons maintenant une *relation de transition* (transition relation) et non plus une *fonction* de transition. Précisément, un automate fini non déterministe est défini par un quintuplet $M = (Q, \Sigma, \Delta, s, F)$, où

- Q est un ensemble d'états,
- Σ est un alphabet,
- $\Delta \subset (Q \times \Sigma^* \times Q)$ est la relation de transition,
- $s \in Q$ est l'état initial,
- $F \subseteq Q$ est l'ensemble des états accepteurs.

Nous avons remplacé la fonction de transition δ par la relation de transition Δ parce que nous voulons laisser la possibilité à un état d'avoir plusieurs successeurs pour une même lettre se trouvant sur le ruban d'entrée. Concrètement, nous voulons par exemple pouvoir exprimer que $\delta(q, a)$ est q_1 ou q_2 . Une fonction ne pouvant avoir qu'une seule valeur pour des arguments donnés, nous devons choisir une autre représentation. Ce que nous faisons est de donner tous les triplets (p, σ, q) tels que, si le caractère indiqué par la tête de lecture est σ , il est possible de passer de l'état p à l'état q . Comme nous voulons aussi permettre des transitions qui ne déplacent pas la tête de lecture ou qui consultent plusieurs caractères du mot d'entrée, nous introduisons dans les triplets, non pas un seul caractère, mais un mot arbitraire $u \in \Sigma^*$.

Pour définir le langage accepté par un automate non déterministe, nous utilisons exactement les mêmes concepts que pour les automates déterministes : configuration, dérivabilité en une étape, dérivabilité en plusieurs étapes, exécution, mot accepté, langage accepté. Les seules définitions que nous devons changer sont celles de dérivabilité en une étape et de mot accepté.

Définition 2.6

La configuration (q', w') est dérivable en une étape de la configuration (q, w) par la machine M ($((q, w) \vdash_M (q', w'))$) si

- $w = uw'$ (le mot w commence par un préfixe $u \in \Sigma^*$),
- $(q, u, q') \in \Delta$ (le triplet (q, u, q') est dans la relation de transition Δ).

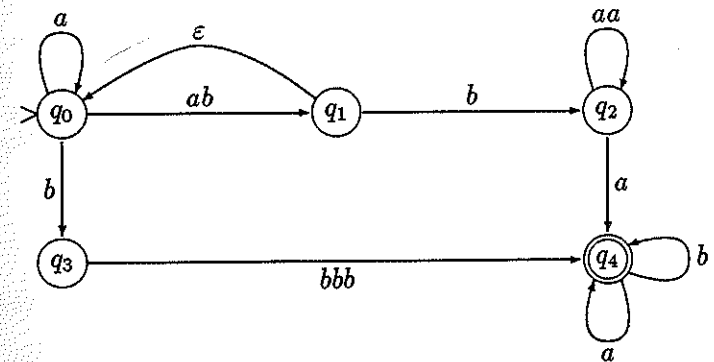
□

La définition de l'exécution d'un automate non déterministe est semblable à celle de l'exécution d'un automate déterministe (définition 2.3), mais cette fois *l'exécution n'est pas unique*. La définition d'un mot accepté est identique à la définition 2.4, mais elle ne nécessite que *l'existence* d'une exécution menant à un état accepteur. Les autres exécutions peuvent très bien mener à des états qui ne sont pas accepteurs.

2.5.3 Exemples

Exemple 2.5

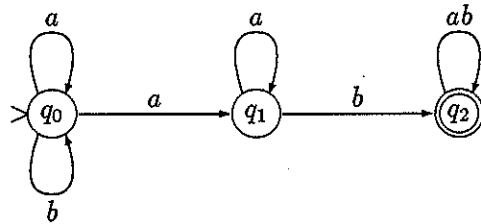
Le langage accepté par l'automate M



est $L(M) = ((a \cup ab)^* bbb \Sigma^*) \cup ((a \cup ab)^* abb(aa)^* a \Sigma^*)$. □

Exemple 2.6

Soit M l'automate ci-dessous.



$L(M) = \Sigma^*ab(ab)^*$, c'est-à-dire l'ensemble des mots se terminant par au moins une répétition de ab . $\square$

Notons que dans un automate non déterministe, la relation de transition n'est pas nécessairement totale. C'est-à-dire que dans chaque état il n'y a pas forcément une transition pour chaque lettre de l'alphabet.

2.6 L'élimination du non-déterminisme

Dans cette section, nous allons démontrer que le non-déterminisme n'ajoute rien à la capacité des automates finis. Pour ce faire, nous allons prouver que l'on peut toujours remplacer un automate fini non déterministe par un automate fini déterministe équivalent. La notion d'équivalence entre automates est naturellement celle de l'égalité des langages acceptés.

Définition 2.7

Deux automates M_1 et M_2 sont *équivalents* s'ils acceptent le même langage, c'est-à-dire si $L(M_1) = L(M_2)$. $\square$

Nous pouvons maintenant énoncer le théorème que nous allons démontrer.

Théorème 2.1

Pour tout automate fini non déterministe, il est possible de construire un automate fini déterministe équivalent. $\square$

Avant de présenter de façon formelle la construction permettant de passer d'un automate non déterministe à un automate déterministe, nous allons en décrire les idées principales.

2.6.1 Principe de la construction

Il y a deux éléments essentiels qui différencient un automate non déterministe d'un automate déterministe :

1. la présence de transitions sur des mots de longueur supérieure à 1,
2. le fait que les transitions possibles pour un même état et même contenu du ruban d'entrée ne sont pas exclusives (autrement dit la présence du non-déterminisme).

Pour obtenir un automate déterministe à partir d'un automate non déterministe, nous commencerons donc par éliminer les transitions sur des mots de longueur supérieure à 1. Cela se fait par une construction schématisée dans la figure 2.2.

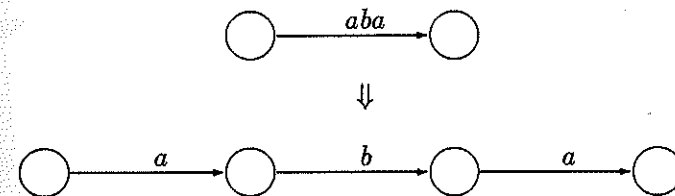


Figure 2.2 - Elimination des transitions de longueur > 1

Le problème des transitions non exclusives est un peu plus délicat. En effet, l'automate déterministe doit d'une façon ou d'une autre tenir compte

de toutes les transitions possibles de l'automate non déterministe. La solution est de construire un automate déterministe qui, à chaque étape, mémorise tous les états dans lesquels l'automate non déterministe pourrait se trouver. Cela implique que chaque état de l'automate déterministe correspondra à un ensemble d'états de l'automate non déterministe.

La figure 2.3 représente un exemple de cette transformation. Dans l'état q_0 et pour la lettre a , l'automate non déterministe peut aller vers l'état q_0 ou vers l'état q_1 . L'automate déterministe, quant à lui, aura une seule transition pour la lettre a . Cette transition mène vers un état étiqueté $\{q_0, q_1\}$ indiquant qu'il correspond à deux états possibles de l'automate non déterministe.

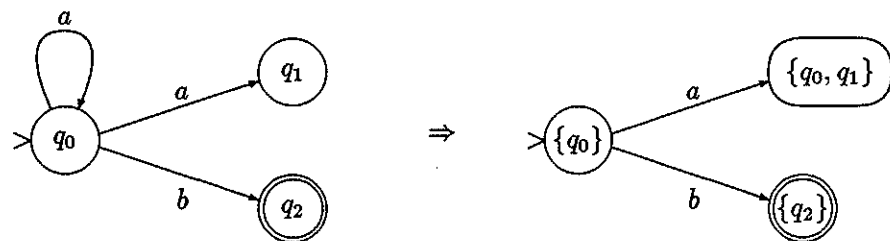


Figure 2.3 - Elimination du non-déterminisme

2.6.2 Formalisation de la construction

Notre construction formelle est divisée en deux étapes. La première est l'élimination des transitions de longueur supérieure à 1.

Elimination des transitions de longueur > 1

Soit un automate non déterministe

$$M = (Q, \Sigma, \Delta, s, F).$$

Nous allons construire un automate non déterministe équivalent

$$M' = (Q', \Sigma, \Delta', s, F)$$

qui ne contient pas de transitions de longueur supérieure à 1, c'est-à-dire tel que

$$\forall (q, u, q') \in \Delta', |u| \leq 1.$$

L'ensemble des états Q' et l'ensemble des transitions Δ' de l'automate M' sont obtenus à partir de ceux de l'automate M comme suit.

- Initialement $Q' = Q$ et $\Delta' = \Delta$.
- Pour chaque transition $(q, u, q') \in \Delta$ avec $u = \sigma_1 \sigma_2 \dots \sigma_k$, ($k > 1$) :
 - on enlève cette transition de Δ' ,
 - on ajoute de nouveaux états $p_1, \dots, p_{k-1}$ à Q' ,
 - on ajoute les nouvelles transitions $(q, \sigma_1, p_1), (p_1, \sigma_2, p_2), \dots, (p_{k-1}, \sigma_k, q')$ à Δ'

Elimination du non-déterminisme

Soit un automate non déterministe

$$M = (Q, \Sigma, \Delta, s, F)$$

où pour toute transition $(q, u, q') \in \Delta$ on a $|u| \leq 1$. C'est-à-dire que chaque transition a soit la forme (q, σ, q') ($\sigma \in \Sigma$), soit la forme (q, ε, q') . Construisons un automate déterministe

$$M' = (Q', \Sigma, \delta', s', F')$$

équivalent à M .

Une des difficultés de cette construction est le traitement des transitions sur le mot vide (celles de la forme (q, ε, q')). En effet, un automate déterministe ne peut avoir de transitions sur le mot vide. Nous devons donc regrouper toute suite de transitions sur le mot vide de l'automate non déterministe avec une transition sur un élément de Σ . Pour ce faire, la définition suivante est utile.

Définition 2.8

Pour un état q d'un automate M , $E(q)$ est l'ensemble des états qui peuvent être atteints à partir de q par une suite de transitions sur le mot vide :

$$E(q) = \{p \in Q \mid (q, w) \vdash_M^* (p, w)\}.$$

□

Notons qu'il est aisé de construire l'ensemble $E(q)$ pour un état q donné : il s'agit de l'ensemble des états p tels que, dans le graphe représentant l'automate, il existe un chemin de q vers p dont tous les arcs sont étiquetés par ϵ .

Définissons maintenant les éléments de l'automate M' .

- L'ensemble des états Q' de M' est l'ensemble des sous-ensembles des états de M . Nous écrirons³

$$Q' = 2^Q.$$

Chaque état de l'automate déterministe correspond donc à un ensemble d'états de l'automate non déterministe.

- L'état initial s' de M' doit représenter les états où l'automate non déterministe M pourrait se trouver avant de commencer à lire son mot d'entrée. Il s'agit donc de tous les états accessibles de l'état initial s de M par une séquence de transitions sur le mot vide. Nous avons donc

$$s' = E(s).$$

- La fonction de transition δ' doit regrouper les transitions possibles de M . Donc, pour un état $\mathbf{q}$ de M' (c'est-à-dire un ensemble d'états q_i de M), $\delta'(\mathbf{q}, a)$ sera l'ensemble des états qui, dans M , peuvent être atteints à partir d'un des états $q_i \in \mathbf{q}$ par une suite de transitions dont la première est étiquetée par a et les suivantes par le mot vide ϵ . Soit

$$\delta'(\mathbf{q}, a) = \bigcup \{E(p) \mid \exists q \in \mathbf{q} : (q, a, p) \in \Delta\}.$$

3. La notation 2^A désigne l'ensemble des sous-ensembles d'un ensemble A . Elle provient de l'expression représentant la cardinalité de cet ensemble : $2^{|A|}$.

- Un état de l'automate déterministe M' sera accepteur s'il contient un état accepteur de l'automate non déterministe M . En effet, si l'automate déterministe se trouve dans un état $\mathbf{q}$, l'automate non déterministe pourrait se trouver dans n'importe quel état $q \in \mathbf{q}$. La définition de F' est donc

$$F' = \{\mathbf{q} \in Q' \mid \mathbf{q} \cap F \neq \emptyset\}.$$

Exemple 2.7

La figure 2.4 représente un automate non déterministe et l'automate déterministe correspondant. □

Dans la figure 2.4, nous n'avons pas représenté tous les 2^5 états de l'automate M' . Nous avons uniquement représenté les états *accessibles* à partir de l'état initial de M' . Intuitivement, un état p est accessible à partir d'un état q s'il existe une exécution qui mène de p à q . De façon plus précise, on obtient la définition suivante.

Définition 2.9

Un état p est *accessible* à partir d'un état q s'il existe un mot w tel que

$$(q, w) \vdash_M^* (p, \epsilon).$$

□

Ou encore, un état p est accessible à partir d'un état q dans un automate M si dans la représentation de M par un graphe, il existe un chemin de q vers p . Un état d'un automate qui n'est pas accessible à partir de l'état initial ne figurera jamais dans une exécution de cet automate. Il est donc sans intérêt et peut être omis sans modification du langage accepté par l'automate.

Notre construction peut donc générer un certain nombre d'états inaccessibles et donc inutiles. Il est possible de la modifier pour que l'ensemble des états Q' de l'automate déterministe ne contienne que des états accessibles. Cela se fait en générant les éléments Q' de proche en proche.

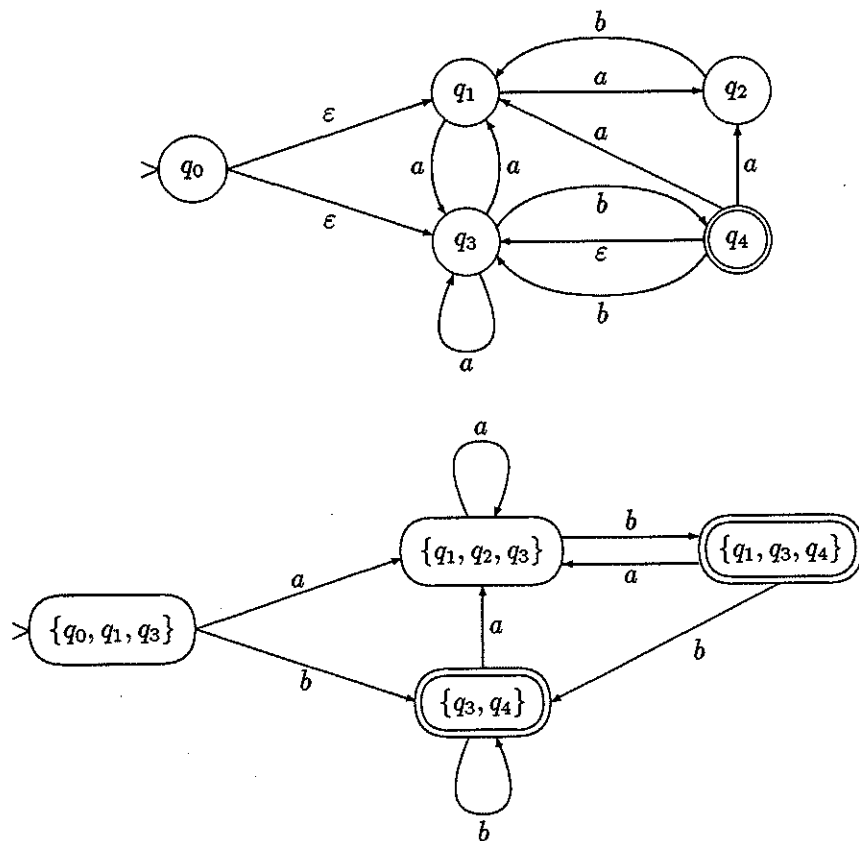


Figure 2.4 - Exemple 2.7

1. Au départ Q' contient l'état initial s' .
2. Les opérations suivantes sont alors répétées jusqu'à ce qu'elles ne modifient plus l'ensemble Q' .
 - (a) On choisit un état $q \in Q'$ auquel l'opération (b) n'a pas encore été appliquée.
 - (b) Pour chaque lettre $a \in \Sigma$ on calcule l'état p tel que $p = \delta'(q, a)$. L'état p est ajouté à Q' .

Exactitude de la construction

Nous avons montré comment construire un automate déterministe à partir d'un automate non déterministe. Il est relativement évident que cette construction produit un automate équivalent à l'automate de départ. En toute rigueur, il faudrait le démontrer.

Que faut-il démontrer exactement? Essentiellement que tout mot accepté par M est accepté par M' et *vice versa*. C'est-à-dire que, pour tout $w \in \Sigma^*$, s'il existe une exécution de M qui accepte w , alors il existe une exécution de M' qui accepte w et *vice versa*.

Considérons un mot arbitraire

$$w = \sigma_1 \dots \sigma_k$$

et commençons par supposer l'existence d'une exécution de M' qui accepte w . Cette exécution a la forme

$$(E(s), w) \vdash_{M'} (q_1, w_1) \vdash_{M'} (q_2, w_2) \dots \vdash_{M'} (q_k, \sigma_k) \vdash_{M'} (p, \varepsilon)$$

avec $p \in F'$. Montrons l'existence d'une exécution de M qui accepte w .

La définition de F' implique que si $p \in F'$, il existe un $p \in Q$ tel que à la fois $p \in p$ et $p \in F$. De plus, le fait que $p \in p$ et la définition de la fonction de transition δ impliquent que si

$$(q_k, \sigma_k) \vdash_{M'} (p, \varepsilon)$$

il existe q_k et $r_k \in Q$ tels que $q_k \in q_k$, $p \in E(r_k)$ et

$$(q_k, \sigma_k, r_k) \in \Delta.$$

Par conséquent, nous avons que

$$(q_k, \sigma_k) \vdash_M^* (p, \varepsilon).$$

Donc il existe une exécution de l'automate M qui, partant de l'état q_k , accepte le mot σ_k . Reprenant le même raisonnement pour la transition

$$(q_{k-1}, \sigma_{k-1}) \vdash_{M'} (q_k, \sigma_k),$$

on peut déduire qu'il existe des états q_{k-1} et $r_{k-1} \in Q$ tels que $q_{k-1} \in q_{k-1}$, $q_k \in E(r_{k-1})$ et

$$(q_{k-1}, \sigma_{k-1}, r_{k-1}) \in \Delta.$$

En continuant un raisonnement identique de proche en proche, on arrive à la conclusion qu'il existe une exécution de M de la forme

$$(s, w) \vdash_M^* \dots \vdash_M^* (q_{k-1}, \sigma_{k-1} \sigma_k) \vdash_M^* (q_k, \sigma_k) \vdash_M^* (p, \varepsilon)$$

et donc que $w \in L(M)$.

Supposons maintenant qu'il existe une exécution de M qui accepte w . Cette exécution peut être écrite sous la forme suivante :

$$\begin{array}{c} \underbrace{(s, w) \vdash_M (q_{01}, w) \vdash_M \dots \vdash_M (q_{0n_0}, w)}_{s, q_{01}, \dots, q_{0n_0} \in E(s) = Q_0} \\ \vdash_M \underbrace{(q_{11}, w_1) \vdash_M (q_{12}, w_1) \vdash_M \dots \vdash_M (q_{1n_1}, w_1)}_{q_{11}, \dots, q_{1n_1} \in \delta(E(s), \sigma_1) = Q_1} \\ \vdash_M \underbrace{(q_{21}, w_2) \vdash_M (q_{22}, w_2) \vdash_M \dots \vdash_M (p, \varepsilon)}_{\substack{q_{21}, \dots, q_{2n_2} \in \delta(Q_1, \sigma_2) = Q_2 \\ p \in \delta(Q_k, \sigma_k) = P}} \end{array} \quad (2.1)$$

Dans cette exécution, nous avons regroupé chaque transition sur un élément de Σ avec toutes les transitions sur le mot vide qui la suivent directement. Faisant cela, il est aisé de voir que l'existence de l'exécution 2.1 de M implique celle de l'exécution suivante de M' :

$$(E(s), w) \vdash_{M'} (q_1, w_1) \vdash_{M'} (q_2, w_2) \vdash_{M'} \dots \vdash_{M'} (p, \varepsilon)$$

2.7 Automates finis et expressions régulières

Nous avons présenté deux formalismes permettant de décrire des langages : les expressions régulières et les automates finis (déterministes et non déterministes). Ces deux formalismes sont de natures très différentes. Les expressions régulières sont basées sur la description d'ensembles de mots à partir d'ensembles élémentaires et d'opérations sur ces ensembles. Par contre, les automates définissent les langages de façon plus opérationnelle, en donnant un mécanisme permettant de les reconnaître.

Il est naturel d'essayer de comparer les classes de langages qui peuvent être définies par ces deux formalismes. Il se fait qu'elles sont identiques ! Cela se traduit par le théorème suivant.

Théorème 2.2

Un langage est régulier si et seulement si il est accepté par un automate fini. $\square$

Pour démontrer ce théorème nous utiliserons le fait qu'un langage est régulier si et seulement si il est décrit par une expression régulière (théorème 1.1). De plus, le théorème 2.1 nous permet de nous limiter aux automates finis non déterministes. Nous allons donc démontrer les deux lemmes suivants.

Lemme 2.1

Si un langage est dénoté par une expression régulière, il est accepté par un automate fini non déterministe. $\square$

Lemme 2.2

Si un langage est accepté par un automate fini non déterministe, il est régulier. $\square$

Considérons d'abord le lemme 2.1.

2.7.1 Des expressions régulières aux automates

Supposons qu'un langage est décrit par une expression régulière et montrons comment construire un automate non déterministe acceptant ce langage. Pour ce faire, nous allons nous servir de la définition des expressions régulières. Nous allons montrer que pour chaque expression régulière de base (ϕ , ε , $a \in \Sigma$) il existe un automate acceptant le même langage et que pour chaque expression régulière composée ($(\alpha_1 \alpha_2)$, $(\alpha_1 \cup \alpha_2)$, $(\alpha_1)^*$) il est possible d'obtenir un automate acceptant le langage décrit par ces expressions à partir d'automates acceptant les langages décrits par α_1 et α_2 .

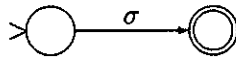
Commençons par donner les automates correspondant aux expressions régulières de base. Pour ϕ , l'automate est



Pour ε , nous utilisons l'automate suivant :



Finalement, l'automate correspondant à $\sigma \in \Sigma$ est



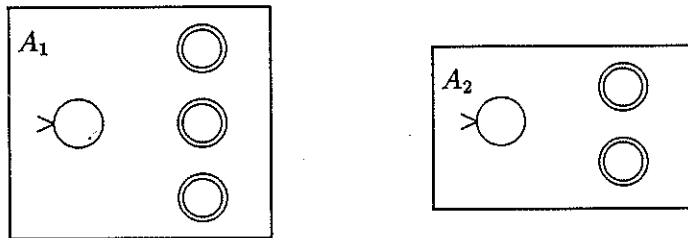
Supposons maintenant que nous ayons déjà construit des automates correspondant aux expressions régulières α_1 et α_2 , soit respectivement

$$A_1 = (Q_1, \Sigma, \Delta_1, s_1, F_1)$$

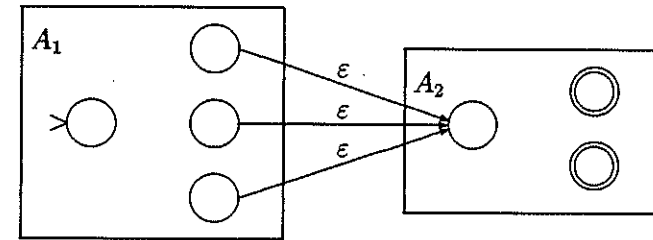
et

$$A_2 = (Q_2, \Sigma, \Delta_2, s_2, F_2).$$

Nous représentons ces automates ci-dessous, mettant uniquement en évidence leur état initial et leurs états accepteurs.



L'automate pour l'expression régulière $\alpha_1 \cdot \alpha_2$ est alors



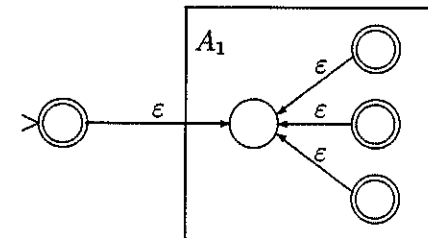
Nous avons obtenu cet automate en reliant tous les états accepteurs de A_1 à l'état initial de A_2 par des transitions ε . L'état initial du nouvel automate est celui de A_1 et ses états accepteurs sont ceux de A_2 . De façon formelle, nous avons construit un automate

$$A = (Q, \Sigma, \Delta, s, F)$$

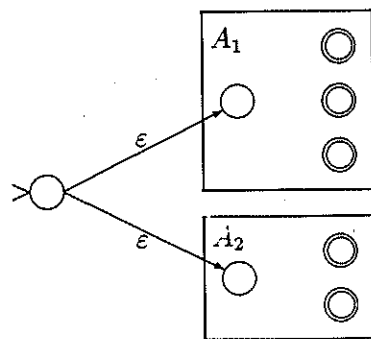
où

- $Q = Q_1 \cup Q_2$,
- $\Delta = \Delta_1 \cup \Delta_2 \cup \{(q, \varepsilon, s_2) \mid q \in F_1\}$,
- $s = s_1$,
- $F = F_2$.

Pour la fermeture itérative ($\alpha = \alpha_1^*$), nous construisons l'automate suivant.



Remarquons que nous avons ajouté un nouvel état servant d'état initial. Cela est nécessaire vu que $\varepsilon \in L(\alpha_1^*)$ et que l'on pourrait ne pas avoir $\varepsilon \in L(\alpha_1)$. Finalement, pour l'opération d'union ($\alpha = \alpha_1 \cup \alpha_2$), nous avons la construction suivante.



Il est intéressant de remarquer que l'utilisation d'automates non déterministes a rendu notre démonstration beaucoup plus simple que si nous ne disposions que d'automates déterministes.

2.7.2 Des automates aux langages réguliers

Considérons un automate M . Nous voulons construire une expression régulière décrivant $L(M)$. Une des premières solutions qui vient à l'esprit pour résoudre ce problème est la suivante :

1. on considère les chemins entre l'état initial de M et les états accepteurs de M .
2. on obtient l'expression régulière correspondant à chacun de ces chemins en concaténant les étiquettes de leurs transitions. Les boucles sont traitées par l'introduction de l'opérateur $*$.
3. l'expression régulière désirée est l'union des expressions régulières ainsi obtenues.

Cette solution est essentiellement celle que nous allons adopter, mais elle comporte un élément imprécis : le traitement des boucles. Il nous faut

une approche plus rigoureuse pour obtenir l'expression régulière correspondant aux chemins existant entre deux états.

Notre solution est de construire cette expression de façon inductive. La seule chose un peu particulière est la nature de notre induction. Supposons que l'ensemble des états de M est $Q = \{q_1, q_2, \dots, q_n\}$. Pour construire l'ensemble des mots permettant de passer d'un état q_i à un état q_j , nous considérons d'abord les mots qui permettent de passer de q_i à q_j sans état intermédiaire ; puis en passant seulement par q_1 ; puis seulement par q_1 et q_2 , ...

Définition 2.10

Soit un automate M et $Q = \{q_1, q_2, \dots, q_n\}$ l'ensemble de ses états. Nous désignons par $R(i, j, k)$ l'ensemble des mots permettant de passer de l'état q_i à q_j en passant uniquement par des états $\{q_1, \dots, q_{k-1}\}$. $\square$

Montrons maintenant comment définir $R(i, j, k)$. Pour $k = 1$, il faut considérer uniquement les mots apparaissant dans les transitions de q_i à q_j ainsi que le mot vide si $i = j$. Nous avons donc

$$R(i, j, 1) = \begin{cases} \{w \mid (q_i, w, q_j) \in \Delta\} & \text{si } i \neq j \\ \{\varepsilon\} \cup \{w \mid (q_i, w, q_j) \in \Delta\} & \text{si } i = j \end{cases}$$

Pour $k > 1$, nous procédons de façon inductive. Les mots qui permettent de passer de q_i à q_j en n'utilisant que des états parmi $\{q_1, q_2, \dots, q_k\}$ sont

- ceux qui permettent de passer de q_i à q_j en n'utilisant que $\{q_1, q_2, \dots, q_{k-1}\}$,
- ceux qui permettent de passer de q_i à q_k , puis de q_k à q_j un certain nombre de fois et finalement de q_k à q_j , le tout en n'utilisant que $\{q_1, q_2, \dots, q_{k-1}\}$ (voir figure 2.5).

Nous avons donc

$$R(i, j, k+1) = R(i, j, k) \cup R(i, k, k)R(k, k, k)^*R(k, j, k)$$

Finalement, le langage accepté par l'automate s'exprime comme étant l'union de tous les ensembles de mots qui permettent de passer de l'état

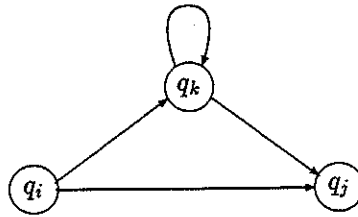


Figure 2.5 - Définition de $R(i, j, k + 1)$

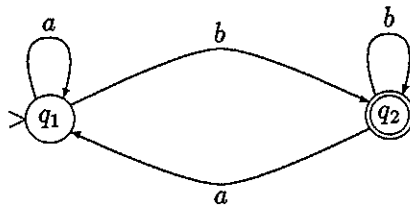
initial à un des états accepteurs, en passant le cas échéant par tous les états de l'automate. En supposant que l'état initial est q_1 et que l'automate comporte n états, nous avons donc

$$L(M) = \bigcup_{q_j \in F} R(1, j, n + 1).$$

Pour définir les langages $R(i, j, k)$ nous n'avons utilisé que des ensembles finis de mots, et les opérations d'union, de concaténation et de fermeture itérative. Ce sont donc des langages réguliers, ce qui établit que $L(M)$ est aussi un langage régulier.

Exemple 2.8

Soit l'automate représenté ci-dessous.



Pour $k = 1$ et $k = 2$, les langages $R(i, j, k)$ obtenus pour cet automate sont les suivants.

	$k = 1$	$k = 2$
$R(1, 1, k)$	$\varepsilon \cup a$	$(\varepsilon \cup a) \cup (\varepsilon \cup a)(\varepsilon \cup a)^*(\varepsilon \cup a)$
$R(1, 2, k)$	b	$b \cup (\varepsilon \cup a)(\varepsilon \cup a)^*b$
$R(2, 1, k)$	a	$a \cup a(\varepsilon \cup a)^*(\varepsilon \cup a)$
$R(2, 2, k)$	$\varepsilon \cup b$	$(\varepsilon \cup b) \cup a(\varepsilon \cup a)^*b$

Le langage accepté par l'automate est alors $R(1, 2, 3)$, soit

$$[b \cup (\varepsilon \cup a)(\varepsilon \cup a)^*b] \cup [b \cup (\varepsilon \cup a)(\varepsilon \cup a)^*b][(\varepsilon \cup b) \cup a(\varepsilon \cup a)^*b]^*[(\varepsilon \cup b) \cup a(\varepsilon \cup a)^*b]$$

□

2.8 Exercices

2.1 Justifier le fait que l'ensemble des automates finis (déterministes ou non déterministes) est un ensemble infini dénombrable.

2.2 Pour chacun des langages suivants, construire un automate fini non déterministe l'acceptant.

1. Les représentations binaires des nombres pairs.
2. Les représentations décimales des multiples de 3.
3. Le langage des mots sur l'alphabet $\{a, b\}$ contenant soit la chaîne aab soit la chaîne $aaab$.

2.3 Construire des automates déterministes pour les langages décrits à l'exercice 2.2.

2.4 Appliquer la construction éliminant le non-déterminisme à l'automate décrit à l'exemple 2.6 (page 50).

2.5 Construire un automate non déterministe acceptant le langage $(aab \cup aa \cup aabb)^*$. Construire un automate déterministe équivalent.

2.6 Pour chacune des expressions régulières suivantes, construire un automate acceptant le langage qu'elle décrit.

1. a^*b

2. $\varepsilon \cup (a \cup aab)^* \cup a^*(aab)^*$

2.7 Construire l'expression régulière dénotant le langage accepté par l'automate décrit à l'exemple 2.4 (page 46).

3

Les grammaires régulières

3.1 Introduction

Dans ce chapitre, nous allons faire une parenthèse dans notre recherche d'une formalisation de la notion de procédure effective. Nous nous intéresserons de plus près à la notion de langage, non pas en tant que formalisation de la notion de problème mais en tant qu'ensemble de mots satisfaisant un certain nombre de règles.

Cette vue du concept de langage a son origine dans des essais de formalisation du langage naturel. Le but (non encore atteint) était de donner une description précise des règles permettant de construire les phrases correctes d'une langue.

Exemple 3.1

Quelques règles permettant de construire des phrases du français sont les suivantes :

- une phrase est de la forme sujet verbe
- un sujet est un pronom
- un pronom est il ou elle
- un verbe est dort ou écoute